Optimization for Machine Learning – Homework Report

Emilio Segovia Salas

Contents

1 Model **2**

2 Gradients **2**

3 Line Search **3**

4 Results **3**

5 Conclusion **3**

1 Model

We start by defining the input and output of the model:

- Input: $x_i \in \mathbb{R}^{256}$
- Output: $y_i \in \mathbb{R}$

The predicted output ($\hat{y}_i$) is defined by the equation

$$\hat{y}_i = W_2\, \sigma(W_1 x_i + b_1) + b_2.$$

The trainable parameters are:

- $W_1 \in \mathbb{R}^{512\times 256}$
- $b_1 \in \mathbb{R}^{512}$
- $W_2 \in \mathbb{R}^{1\times 512}$
- $b_2 \in \mathbb{R}$

The activation function, $\sigma()$ is given by the equation

$$\sigma(z) = \max(0, z)$$

2 Gradients

We use the mean squared error (MSE) loss

$$L = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2.$$

The derivative of the loss with respect to the prediction is

$$\frac{\partial L}{\partial \hat{y}_i} = \frac{2}{N}(\hat{y}_i - y_i).$$

Using the chain rule, the gradients of the output layer parameters are

$$\frac{\partial L}{\partial W_2} = \left(\frac{2}{N}(\hat{y} - y)\right)^{\top} S, \qquad \frac{\partial L}{\partial b_2} = \sum_{i=1}^{N}\frac{2}{N}(\hat{y}_i - y_i).$$

For the hidden layer, we propagate through the ReLU activation

$$\frac{\partial S}{\partial Z} = \mathbf{1}(Z > 0),$$

so the gradients become

$$\frac{\partial L}{\partial W_1} = \left(\frac{2}{N}(\hat{y} - y)W_2\right)^{\top} X,$$

$$\frac{\partial L}{\partial b_1} = \sum_{i=1}^{N}\left(\frac{2}{N}(\hat{y}_i - y_i)W_2\right) \odot \mathbb{1}(Z_i > 0).$$

These analytical gradients are used during training to update all parameters.

3 Line Search

We can generalize a set of equation to calculate the new values for the trainable equations

$$W_{1+1} = W_1` + t\partial W_1$$

$$W_{2+1} = W_2` + t\partial W_2$$

$$b_{1+1} = b_1` + t\partial b_1$$

$$b_{2+1} = b_2` + t\partial b_2$$

In these equations we see a constant t called the learning rate, for this report we have chosen to use the backtracking line search. To do this we select $\alpha \in (0, 0.5), \beta \in (0, 1)$ for this example we have chosen values of $\alpha = 10^{-4}$ and $\beta = 0.5$ and $t = 1$.

1. We assign $t_{n+1} = t_n * \beta$
2. We evaluate $f(x + \Delta x) < f(x) + \alpha t \nabla f(x)^T \Delta x$

We reiterate 1 and 2 until 2 is true. The search then stops if t is small enough.

4 Results

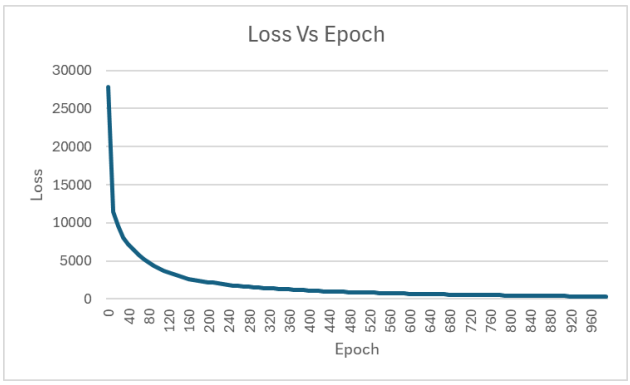


Figure 1: Loss Vs Gain of the algorithm

As all the parameters are getting optimized the loss is reduced exponentially. This trend can clearly be seen from the Figure 1.

5 Conclusion

To finished this report I would like to highlight a few things. The first and main one is the trend shown in my graph is the adequate one that the network should follow. However, there are a few things that require improvement. The first is that even after 1000 epoches the final loss value is still over 300 which is still very large. The reason for this high loss is because the way I calculate the loss is very inefficient. I create really big matrices which requires really inefficient computation. Secondly, training these 4 variables is very time consuming. This is because every epoch requires a new calculation for the learning rate which takes a really big loop, this reapeted for every variable ends up being a very time consuming algorithm.